Contents

1 Project Overview

Project Name: Apex — Resilience First Monitoring Engine
Goal: Performance-critical, offline-first, privacy-preserving crash & error observability platform
Core Philosophy: Zero-data-cost in high-cost regions + tactical AI forensics
Languages / Tech: Go (core), Python & Node.js (agents), Next.js (dashboard), CockroachDB, Redis, Prometheus/Grafana, Gemini AI

1.1 Three Core Pillars

1. **Agent** — panic interception, zero-allocation capture
2. **Vault** — encrypted local SQLite storage (offline resilience)
3. **Syphon** — smart network-aware sync + Zstandard compression

2 Architecture & Data Flow

- **Protobuf** is used as wire & storage format (small binary size)
- Agent captures panic → writes encrypted to Vault (SQLite)
- Syphon batches + compresses → sends only on Wi-Fi (or when allowed)
- Receiver (Go server) → Redis buffer → CockroachDB → AI analysis → Dashboard

3 Development History & Lessons Learned

3.1 Project Initialization

```
go mod init github.com/apex/monitor
mkdir pkg/agent pkg/vault pkg/syphon proto cmd/server cmd/inspector
```

3.2 Lesson 1 — Go Modules & Folder Conventions

Module path = unique ID (even before publishing).
Common layout: `pkg/` for libraries, `cmd/` for executables.

3.3 Lesson 2 — Why Protobuf instead of JSON?

JSON: human-readable but verbose.
Protobuf: binary, 3–10× smaller → critical for expensive mobile data.

3.4 Lesson 3 — Protobuf Toolchain

```
# Install protoc & Go plugin
winget install protobuf
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest

# Generate Go code
protoc --go_out=. --go_opt=paths=source_relative proto/apex.proto
```

3.5 Lesson 4 — Dependency Management

```
go get google.golang.org/protobuf/proto
go mod tidy
```

3.6 Lesson 5 — Panic Recovery in Go

Key idioms:

- `defer func() { if r := recover(); r != nil { ... } }`
- Zero-allocation paths during crash handling

3.7 Lesson 6 — Go Strictness

Unused imports/variables → compile error (keeps code clean).

3.8 Lesson 7 & 18 — runtime Package

- `runtime.GOOS`, `runtime.GOARCH`
- `runtime.ReadMemStats(&m)` → memory diagnostics

3.9 Lesson 12 & 13 — Zstandard Compression

Much better speed/ratio than gzip → ideal for mobile/offline-first.

```
go get github.com/klauspost/compress/zstd
```

Compression results example: 4608 bytes → 728 bytes (84% savings)

3.10 Lesson 17 — AES-GCM Encryption

Local vault security — nonce + authenticated encryption.

3.11 Lesson 15 — Interfaces for Testability

```
type NetworkStatus interface {
    CurrentType() string
}
func (s *Syphon) ShouldSync(status NetworkStatus) bool { ... }
```

3.12 Lesson 21 — Goroutines & Channels

Background sync loop + graceful shutdown via `stopChan`.

3.13 Lesson 20 — cmd/ Pattern

Separate CLIs + servers:

- `cmd/server/main.go` — production receiver
- `cmd/inspector/main.go` — local encrypted DB viewer

4 Infrastructure Evolution

- SQLite (local vault) → Postgres → CockroachDB (distributed)
- Added Redis Streams → durable high-throughput buffer
- Prometheus metrics + Grafana dashboards
- Docker Compose → one-command full stack

5 Dashboard & Frontend (Next.js)

Theme: Amber Industrial / Cyber-Black
Features:

- Real-time crash cards (OS, RAM, battery, AI forensics)
- Project-based isolation & API key management
- Chat with Gemini-powered tactical AI

6 AI Forensics (Gemini Integration)

- Root-cause detection (nil pointers, leaked resources, ...)
- Cached insights (fingerprint = SHA-256 of stack + message)
- Rate limiting per project (Redis sliding window)

7 Multi-Language Agents

- Go — native panic recovery
- Python — exception hook + zstd + requests
- Node.js — unhandled exception handler

8 Final Status — Enterprise Grade

- Self-hostable (Docker Compose)
- 100% open source plan
- Global scaling architecture (CockroachDB + Redis)
- AI tactical root-cause + fix suggestions

Vital Commands (one-click launch):

```
run.bat          # or docker compose up -d
```

Important URLs (local dev):

- Dashboard: http://localhost:3000
- Grafana: http://localhost:3001
- CockroachDB console: http://localhost:8082